

Domain-Driven Design: é uma técnica que consolida um conjunto de práticas aplicadas de forma organizada e sistemática.

Tem como ideia apresentar um conjunto de princípios para projeto e desenvolvimento de sistemas, como foco no domínio da aplicação.

Domínio: Consiste na área e problema de negócio que ele pretende resolver. Como **desenvolvedor**, deve-se ter profundo conhecimento sobre esse domínio, para isso, é necessário conversar com pessoas **especialistas nesse domínio**.

Modelos/abstrações do domínio: criação de modelos a partir do conhecimento adquirido com os especialistas de domínio.

Linguagem Ubíqua: Conjunto de termos que devem ser plenamente entendidos tanto pelos desenvolvedores quanto para os especialistas de domínio, permitindo uma comunicação clara e fluida para ambos.

DDD está bastante alinhado à OO.

OO faz bastante abstrações do mundo real em um sistema de software, tal qual o DDD também a partir da criação de modelos que facilitam a criação de um software.

Desenvolvimento dirigido a modelos:

Entidade: Objeto que possui identidade única, que o distingue dos demais objetos da mesma classe. (Ex.: aluno em uma universidade possui RGA)

Objetos de valor: Não possui identificador único, caracterizados apenas pelos valores de seus atributos. (Ex.: endereço de um aluno, não há como identificar unicamente o endereço, porém é uma informação importante. Sendo assim, um objeto de valor imutável.)

Objetos de valor não existem independentemente e sempre estarão associados a entidades

Agregados: conjuntos de entidades e objetos de valor associados que devem trabalhar juntos (Ex.: venda (entidade) e itens de venda (objetos de valor)).

Serviços: Operações que não se encaixam em entidades e objetos de valor, são objetos comumente associados e referenciados como **controladores** ou gerenciadores. Não devem possuir estados (**stateless**) e normalmente implementados como **singletons**.

Repositórios: criar abstrações que ficam entre a aplicação e o banco de dados, sendo responsável por transformar os dados que quer-se salvar em algo persistente. O usuário acessa o repositório, que fica responsável pelo acesso ao banco.

Domínios complexos e integridade: à medida que ocorrem mudanças e adaptações nos modelos, é muito fácil que os modelos possam se deteriorar ao longo do tempo.

Para evitar que isso ocorra, existem técnicas que podem ser aplicadas buscando manter a integridade deste modelo.

Contextos delimitados: consiste em quebrar domínios complexos em pequenos domínios.

Importante lembrar: ao quebrar o domínio em pequenos domínios, eles ainda devem se comunicar entre si. Sendo assim, é importante **manter a consistência** entre esses domínios através da criação de **mapas de contextos**, associando elementos de um contexto A com um contexto B.

Núcleos compartilhados: modelos que são compartilhados por dois contextos delimitados. Devem ser mantidos pelas equipes, não podendo ser modificado sem a aprovação de todos que utilizam aquele modelo.

Camada Anticorrupção: Ajuda a mediar a comunicação entre contextos delimitados diferentes. Utilizam de padrões de projeto como serviços, adaptadores, fachadas.

DDD é uma **abordagem** para desenvolvimento de sistemas **complexos**, com **foco no domínio** do sistema; **desenvolvedores e especialistas** devem explorar o domínio de forma colaborativa, utilizando da **linguagem ubíqua** dentro de um **contexto delimitado**.

Padrões de Projeto: Intimamente ligados à OO, são soluções prontas e elegantes já testadas e aprovadas para solucionar problemas que ocorrem frequentemente na programação.

Principais Design Patterns

Padrões de Criação: tratam da criação de objetos.

- **Singleton: centraliza e compartilha recursos.**
- Abstract Factory: cria famílias de objetos com alta flexibilidade.
- Builder: constrói o produto passo a passo.
- Factory: cria objetos com alta flexibilidade.
- Prototype: cria objetos por cópia de uma instância.

Padrões Estruturais: cuidam da composição de classes e objetos.

- **Facade: simplifica a utilização de subsistemas complexos.**
- **Adapter: pluga o conteúdo ao sistema.**
- **Decorator: incrementa funcionalidades dinamicamente.**

- Bridge: separa implementações de abstrações em prol da flexibilidade.
- Composite: compõe objetos em estruturas de árvores e trata como objetos individuais.
- Flyweight: compartilha de pequenos recursos para economizar espaço.
- Proxy: faz um objeto representar outro objeto.

Padrões Comportamentais: Caracterizam o modo como as classes e objetos interagem e distribuem responsabilidades.

- **Observer: faz o compartilhamento de recursos de forma inteligente.**
- **Strategy: separa os dados dos algoritmos para alcançar a reusabilidade.**
- **Command: transforma requisições em objetos.**
- **Iterator: percorre um conjunto de dados independente da implementação.**
- Interpreter: define uma gramática e um interpretador.
- Chain of responsibility: repassa requisições para evitar decisões.
- Mediator: simplifica relacionamentos complexos.
- Memento: externaliza status sem quebrar o encapsulamento.
- State: simplifica a troca de estados internos de um objeto.
- Template: define algoritmos extensíveis.
- Visitor: define uma nova operação para uma classe sem alterá-la.

Data Access Object - DAO: padrão utilizado para abstrair e encapsular todos os acessos ao data source, gerenciando a conexão para obter e armazenar informações.

DAO não é um padrão de criação, seu objetivo é fazer a persistência de dados em um data source.

Elementos essenciais de um padrão:

- **Nome do padrão:** nome ao qual referenciamos o padrão em uma ou duas palavras.
- **Problema:** Explica o problema e contexto para que possamos identificar quando aplicá-lo.
- **Solução:** Descreve os elementos da solução, relacionamentos, responsabilidades e colaborações.
- **Consequências:** Os resultados e trade-offs aplicados ao padrão.

Vantagens: Aumento da produtividade; Facilitam o reuso de técnicas já aplicadas com sucesso; Redução da complexidade: blocos construtivos; Uniformidade na estrutura do software.

Desvantagens: Necessita ser reprogramado em cada aplicação; Uso excessivo pode aumentar muito a complexidade.

EXTRA:

Singleton: Garantir que uma classe tenha somente uma instância e fornecer um ponto de acesso global de acesso para a mesma.

Facade: Fornecer uma interface unificada para um conjunto de interfaces em um subsistema.

Adapter: Converter a interface de uma classe em outra interface esperada pelos clientes.

Strategy: Definir uma família de algoritmos, encapsular cada uma delas e torná-las intercambiáveis. Permite que o algoritmo varie independentemente dos clientes que o utilizam.

Decorator: Dinamicamente, agregar responsabilidades adicionais a um objeto.

Observer: Definir uma dependência um-para-muitos entre objetos, de maneira que quando um objeto muda de estado, todos os seus dependentes são notificados e atualizados automaticamente.

Command: Encapsular uma requisição como um objeto independente que contém toda a informação sobre o pedido, permitindo diferentes requisições, enfileiramento, logs e suporte operações que não podem ser feitas.

Iterator: Prover uma forma de acesso a elementos de algum conjunto de dados de forma sequencial sem expor sua representação interna.